

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Sistemas Distribuidos				
TÍTULO DE LA PRÁCTICA:	SOAP Web services				
NÚMERO DE LA PRÁCTICA:	07	AÑO LECTIVO:	2026	NRO. SEMESTRE:	A
FECHA DE PRESENTACIÓN:	02/06/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(s): - Bedregal Perez Daniel - Jara Mamani Mariel Alisson - Mestas Zegarra Christian Raul - Noa Camino Yenaro Joel - Sequeiros Condori Luis Gustavo				NOTA (0 - 20):	Nota colocada por el docente
DOCENTE: Mg. Maribel Molina Barriga					

RESULTADOS Y PRUEBAS
ENLACE A GITHUB https://github.com/christianmz565/sd-2026-lab-c-grupo-3/tree/main/l7
EJERCICIOS RESUELTOS POR EL DOCENTE Ejercicio Resuelto: Servicio SOAP de Suma Siguiendo las instrucciones de la guía, se implementó un servicio web SOAP básico en Java utilizando JAX-WS. El servicio define una operación sumar que recibe dos enteros y retorna su suma. Se definió primero una interfaz para el servicio, asegurando que el contrato sea claro y desacoplado de la implementación: <pre>@WebService(targetNamespace = "http://lab7.docente/") public interface CalculadoraAPI { @WebMethod public int sumar(int a, int b); }</pre> La implementación de dicha interfaz se decoró con la anotación @WebService, especificando el espacio de nombres y los nombres de puerto y servicio para cumplir con los estándares WSDL: <pre>@WebService(endpointInterface = "s1.CalculadoraAPI", serviceName = "CalculadoraSOAPService", portName = "CalculadoraSOAPPort", targetNamespace = "http://lab7.docente/") public class CalculadoraSOAP implements CalculadoraAPI { @Override public int sumar(int a, int b) { return a + b; } }</pre> Para exponer el servicio en la red, se utilizó la clase Endpoint.publish, asignando el servicio a la dirección http://localhost:8080/calculadora:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

```
public class Publicador {

    public static void main(String[] args) {
        Endpoint.publish(
            "http://localhost:8080/calculadora",
            new CalculadoraSOAP()
        );
        System.out.println("Servicio SOAP activo");
    }
}
```

Finalmente, se desarrolló un cliente consumidor en Java que localiza el WSDL, crea un proxy dinámico del servicio y realiza una llamada remota de prueba:

```
public class ClienteSOAP {

    public static void main(String[] args) throws Exception {
        URL url = new URI("http://localhost:8080/calculadora?wsdl").toURL();
        QName qname = new QName("http://lab7.docente/", "CalculadoraSOAPService");
        Service service = Service.create(url, qname);
        CalculadoraAPI calc = service.getPort(CalculadoraAPI.class);
        System.out.println(calc.sumar(10, 20));
    }
}
```

La validación del servicio se muestra en la siguiente captura, donde el cliente recibe correctamente el resultado de la operación remota:



```
> mvn clean compile exec:exec -Dexec.mainClass="s1.ClienteSOAP" -q
30
>
```

Figura 1: Ejecución del cliente SOAP en Java para la operación de suma.

SOLUCIÓN DE EJERCICIOS PROPUESTOS

Ejercicio 1: Servicio SOAP de Conversión de Unidades

Se desarrolló un servicio SOAP más robusto que permite realizar diversas conversiones de unidades: temperatura (Celsius/Fahrenheit), longitud (Metros/Pies) y masa (Kilogramos/Libras). La lógica de negocio se centralizó en la implementación del servicio:

```
@WebService(
    endpointInterface = "e1.ConversorAPI",
    serviceName = "ConversorSOAPService",
    portName = "ConversorSOAPPort",
    targetNamespace = "http://lab7.e1.conversor/"
)
public class ConversorSOAP implements ConversorAPI {

    @Override
    public double cToF(double c) {
        return ((c * 9.0) / 5.0) + 32.0;
    }

    @Override
    public double fToC(double f) {
        return ((f - 32.0) * 5.0) / 9.0;
    }

    @Override
    public double mToFt(double m) {
        return m * 3.28084;
    }

    @Override
    public double ftToM(double ft) {
        return ft / 3.28084;
    }
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

```
@Override
public double kgToLb(double kg) {
    return kg * 2.20462;
}

@Override
public double lbToKg(double lb) {
    return lb / 2.20462;
}
}
```

Para interactuar con este servicio, se implementó un cliente en Python utilizando la librería zeep. Este cliente ofrece un menú interactivo y maneja la comunicación SOAP de forma transparente:

```
def main():
    print(f"\nConectando a: {WSDL_URL}")
    try:
        client = Client(WSDL_URL)
        print("Conexión establecida con éxito.\n")
    except Exception as e:
        print(f"Error al conectar: {e}")
        return

    while True:
        print("---- CONVERSION DE UNIDADES ----")
        # ... (opciones del menú)
        opcion = input("\nSeleccione una opción (1-7): ").strip()

        if opcion == "1":
            valor = float(input("Grados Celsius: "))
            try:
                resultado = client.service.cToF(valor)
                print(f"Resultado: {valor}°C = {resultado:.2f}°F")
            except Exception as e:
                print(f"Error: {e}")
```

A continuación se presentan las evidencias de la ejecución del cliente Python interactuando con el servidor SOAP en Java:

```
> uv run client.py
Conectando a: http://localhost:8080/conversor?wsdl
Conexión establecida con éxito.

---- CONVERSION DE UNIDADES ----
1. Celsius a Fahrenheit
2. Fahrenheit a Celsius
3. Metros a Pies
4. Pies a Metros
5. Kilogramos a Libras
6. Libras a Kilogramos
7. Salir
Seleccione una opción (1-7): 7
¡Hasta luego!
```

Figura 2: Menú interactivo del cliente Python.

```
> uv run client.py
Conectando a: http://localhost:8080/conversor?wsdl
Conexión establecida con éxito.

---- CONVERSION DE UNIDADES ----
1. Celsius a Fahrenheit
2. Fahrenheit a Celsius
3. Metros a Pies
4. Pies a Metros
5. Kilogramos a Libras
6. Libras a Kilogramos
7. Salir
Seleccione una opción (1-7): 1
Grados Celsius: 25
Resultado: 25.0°C = 77.00°F

---- CONVERSION DE UNIDADES ----
1. Celsius a Fahrenheit
2. Fahrenheit a Celsius
3. Metros a Pies
4. Pies a Metros
5. Kilogramos a Libras
6. Libras a Kilogramos
7. Salir
Seleccione una opción (1-7): 7
¡Hasta luego!
```

Figura 3: Resultado de conversiones de temperatura y longitud.

Ejercicio 2: Servicio SOAP de Gestión de Tienda (CRUD)

Se implementó un sistema de inventario avanzado mediante SOAP, permitiendo operaciones CRUD (Create, Read, Update, Delete) y una operación especial de compra que gestiona el stock de los productos.

La arquitectura se basa en un contrato de servicio definido mediante la interfaz S0API, la cual expone las operaciones necesarias para la gestión de productos. Se hace uso de objetos Item que JAX-WS serializa automáticamente a XML:

```
@WebService(targetNamespace = "http://lab7.e2.servicio.soap/")
public interface S0API {

    @WebMethod
    public List<Item> getItems();

    @WebMethod
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

```

public String buyItem(String name, int cantidad);

@WebMethod
public boolean addItem(Item item);

@WebMethod
public boolean setItem(String name, int cantidad, double precio);

@WebMethod
public boolean deleteItem(String name);
}

```

La implementación del servicio en Java (SOAPImpl) delega la lógica de persistencia y reglas de negocio (como la validación de stock insuficiente durante una compra) a la clase de modelo Item:

```

@WebService(
    endpointInterface = "e2.servicio.soap.SOAPI",
    serviceName = "SOAPImplService",
    portName = "SOAPImplPort",
    targetNamespace = "http://lab7.e2.servicio.soap/"
)
public class SOAPImpl implements SOAPI {

    @Override
    public List<Item> getItems() {
        return Item.getItems();
    }

    @Override
    public String buyItem(String name, int cantidad) {
        return Item.buyItem(name, cantidad);
    }

    @Override
    public boolean addItem(Item item) {
        return Item.addItem(item);
    }

    @Override
    public boolean setItem(String name, int cantidad, double precio) {
        return Item.setItem(name, cantidad, precio);
    }

    @Override
    public boolean deleteItem(String name) {
        return Item.deleteItem(name);
    }
}

```

La clase Item define los atributos de un producto (nombre, cantidad, costo) y implementa métodos para la gestión de inventario con código estándar de Java:

```

public class Item implements Serializable {
    private String nombre;
    private int cantidad;
    private double costo;

    private static final List<Item> ITEMS = new ArrayList<>(
        Arrays.asList(
            new Item("Gaseosa", 15, 5.2),
            new Item("Galletas", 10, 1.6),
            new Item("Celular", 12, 900.0)
        )
    );

    public static String buyItem(String name, int cantidad) {
        for (Item item : ITEMS) {
            if (item.nombre.equalsIgnoreCase(name)) {
                if (item.cantidad - cantidad < 0) {
                    return "Not enough items";
                }
            }
        }
    }
}

```

```

    }
    item.cantidad -= cantidad;
    return "Producto: " + item.nombre + " Total: " + (cantidad * item.costo);
  }
  return "Item no match";
}
}

```

Al igual que en el ejercicio anterior, se desarrolló un cliente CLI en Python para validar las operaciones administrativas. En las capturas se observa la creación, actualización y eliminación de productos, así como la persistencia en el servidor:

```

> uv run client.py
Conectando a: http://localhost:1516/WS/Store?wsdl
Conexión establecida con éxito.

--- GESTIÓN DE TIENDA ---
1. Ver inventario
2. Comprar producto
3. Agregar nuevo producto
4. Actualizar producto
5. Eliminar producto
6. Salir

Seleccione una opción (1-6): 3

--- NUEVO PRODUCTO ---
Nombre: Leche
Stock inicial: 50
Precio: 4.5
Producto 'Leche' agregado correctamente.

--- GESTIÓN DE TIENDA ---
1. Ver inventario
2. Comprar producto
3. Agregar nuevo producto
4. Actualizar producto
5. Eliminar producto
6. Salir

Seleccione una opción (1-6): 6
(hasta luego!)
>

```

Figura 4: Agregando un nuevo producto via CLI.

```

> uv run client.py
Conectando a: http://localhost:1516/WS/Store?wsdl
Conexión establecida con éxito.

--- GESTIÓN DE TIENDA ---
1. Ver inventario
2. Comprar producto
3. Agregar nuevo producto
4. Actualizar producto
5. Eliminar producto
6. Salir

Seleccione una opción (1-6): 2

--- COMPRA ---
Nombre del producto: Gaseosa
Cantidad a comprar: 1
Resultado: Producto: Gaseosa Cantidad: 1 Total: 5.20

--- GESTIÓN DE TIENDA ---
1. Ver inventario
2. Comprar producto
3. Agregar nuevo producto
4. Actualizar producto
5. Eliminar producto
6. Salir

Seleccione una opción (1-6): 6
(hasta luego!)
>

```

Figura 5: Simulación de compra de un producto via CLI.

```

> uv run client.py
Conectando a: http://localhost:1516/WS/Store?wsdl
Conexión establecida con éxito.

--- GESTIÓN DE TIENDA ---
1. Ver inventario
2. Comprar producto
3. Agregar nuevo producto
4. Actualizar producto
5. Eliminar producto
6. Salir

Seleccione una opción (1-6): 4

--- ACTUALIZAR PRODUCTO ---
Nombre del producto a actualizar: Pan
Nuevo stock: 200
Nuevo precio: 0.8
Producto 'Pan' actualizado correctamente.

--- GESTIÓN DE TIENDA ---
1. Ver inventario
2. Comprar producto
3. Agregar nuevo producto
4. Actualizar producto
5. Eliminar producto
6. Salir

Seleccione una opción (1-6): 6
(hasta luego!)
>

```

Figura 6: Actualizando stock y precio de un producto.

```

> uv run client.py
Conectando a: http://localhost:1516/WS/Store?wsdl
Conexión establecida con éxito.

--- GESTIÓN DE TIENDA ---
1. Ver inventario
2. Comprar producto
3. Agregar nuevo producto
4. Actualizar producto
5. Eliminar producto
6. Salir

Seleccione una opción (1-6): 1

Producto | Stock | Precio ($/.)
Gaseosa | 15 | 5.20
Café | 10 | 1.40
Celular | 9 | 900.00
Masa | 2 | 2.00
Pan | 120 | 0.30

--- GESTIÓN DE TIENDA ---
1. Ver inventario
2. Comprar producto
3. Agregar nuevo producto
4. Actualizar producto
5. Eliminar producto
6. Salir

Seleccione una opción (1-6): 6
(hasta luego!)
>

```

Figura 7: Listado de productos disponibles en el inventario.

Para el consumo web, debido a las limitaciones del navegador para realizar peticiones SOAP directas (CORS, parsing XML complejo), se implementó un proxy en Node.js (Fastify) que traduce peticiones REST a llamadas SOAP internas:

```

const SOAP_URL = "http://localhost:1516/WS/Store?wsdl";

fastify.get("/api/items", async () => {
  const client = await createClientAsync(SOAP_URL);
  const [result] = await client.getItemsAsync({});
  return normalizeItems(result);
});

fastify.post("/api/items/:nombre/buy", async (request) => {
  const { nombre } = request.params;
  const { cantidad } = request.body || {};
  const client = await createClientAsync(SOAP_URL);
  const [result] = await client.buyItemAsync({
    arg0: nombre,
    arg1: cantidad,
  });
  return { result: result?.return ?? "" };
});

```

Finalmente, se diseñó una interfaz web moderna que permite a los usuarios finales ver el catálogo y realizar compras de forma intuitiva, interactuando indirectamente con el servicio SOAP original:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

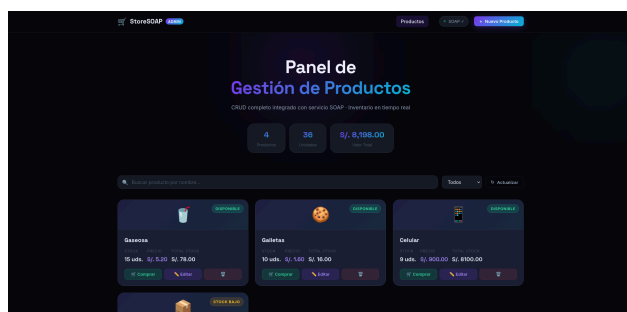


Figura 8: Interfaz web del catálogo de productos.

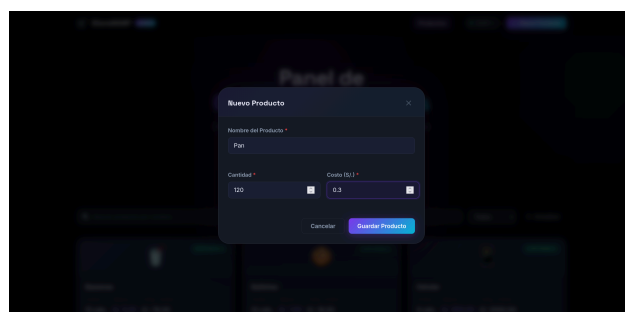


Figura 9: Formulario modal para agregar productos.

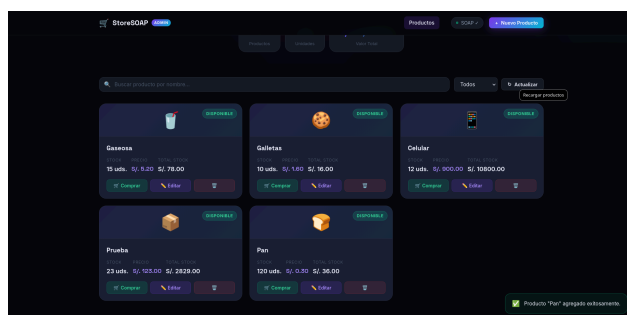


Figura 10: Resultado de la sincronización con el servidor.

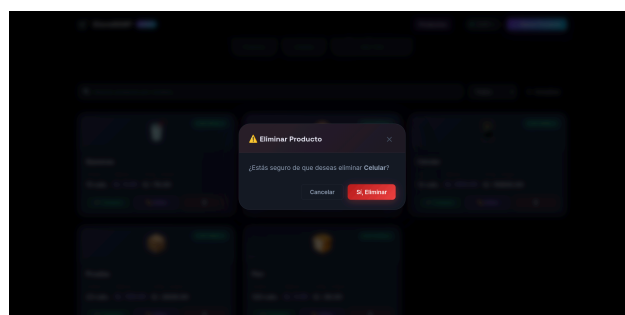


Figura 11: Confirmación de eliminación de recursos en la web.

CUESTIONARIO

1. ¿Puedo utilizar un componente JavaBeans para implementar un servicio web utilizando la invocación de SOAP sobre JMS (Java Message Service)?

Sí, es posible. JAX-WS permite utilizar JavaBeans como implementaciones de servicios web. Al integrar SOAP con JMS, el componente JavaBean actúa como el receptor de los mensajes que llegan a una cola o tópico. El contenedor de aplicaciones se encarga de extraer el cuerpo SOAP del mensaje JMS, deserializarlo e invocar el método correspondiente en el JavaBean, permitiendo una comunicación asíncrona y fiable.

2. ¿Cómo funciona la mensajería bidireccional con la implementación de SOAP y JMS? ¿Da soporte a varios clientes realizando solicitudes simultáneas?

La mensajería bidireccional (solicitud-respuesta) en SOAP sobre JMS se logra mediante el uso de colas de respuesta (ReplyTo). El cliente envía un mensaje a una cola de solicitudes e incluye un identificador de correlación y la dirección de su propia cola de respuesta. El servidor procesa la solicitud y envía la respuesta a la cola especificada con el mismo ID de correlación. Sí da soporte a múltiples clientes simultáneos, ya que JMS es inherentemente escalable y maneja la concurrencia a través de gestores de colas que pueden distribuir mensajes entre múltiples hilos o instancias de servidor.

3. ¿Por qué SOAP sigue siendo utilizado en sistemas empresariales críticos pese al auge de REST?

SOAP prevalece en entornos empresariales (especialmente banca y seguros) debido a su robustez y formalidad. Ofrece estándares estrictos como WS-Security para cifrado y firma digital a nivel de mensaje (no solo transporte), WS-AtomicTransaction para transacciones distribuidas complejas, y el contrato WSDL que garantiza una tipificación fuerte y generación automática de clientes, lo cual reduce errores de integración en sistemas legados y de gran escala.

4. ¿Qué implicancias tiene el uso de XML en el rendimiento de sistemas distribuidos de alta concurrencia?

XML es un formato verboso y basado en texto, lo que implica un mayor consumo de ancho de banda en comparación con formatos binarios o JSON. Además, el parseo de XML requiere más recursos de CPU y memoria (especialmente con DOM), lo que puede

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

convertirse en un cuello de botella en sistemas de alta concurrencia. El uso de SAX o StAX mitiga esto, pero el overhead de serialización/deserialización sigue siendo superior al de alternativas más ligeras.

5. ¿En qué escenarios arquitectónicos SOAP resulta más adecuado que REST? Justifique técnicamente.

SOAP es superior en escenarios que requieren:

- **Seguridad avanzada:** Cuando se necesita seguridad a nivel de mensaje (extremo a extremo) que persista a través de múltiples intermediarios.
- **Transaccionalidad:** En flujos financieros que exigen cumplimiento de propiedades ACID en múltiples servicios distribuidos.
- **Protocolos no-HTTP:** Cuando el servicio debe exponerse sobre SMTP, TCP o colas de mensajes (JMS) de forma transparente.
- **Contratos estrictos:** En integraciones B2B complejas donde la validación estricta de esquemas es crítica para la integridad de los datos.

CONCLUSIONES Y RECOMENDACIONES

CONCLUSIONES

1. SOAP ofrece una arquitectura altamente estructurada y basada en contratos que facilita la interoperabilidad entre diferentes lenguajes (Java y Python en este laboratorio) mediante el uso de WSDL.
2. A pesar de su mayor overhead en comparación con REST, SOAP proporciona capacidades críticas para entornos empresariales, como la seguridad robusta y el soporte para diversos protocolos de transporte.
3. La implementación de servicios SOAP en Java se ha simplificado significativamente con el estándar JAX-WS, permitiendo convertir POJOs en servicios web mediante anotaciones simples.

RECOMENDACIONES

1. Se recomienda utilizar herramientas como SOAPUI o Postman para la inspección y depuración de los mensajes XML generados, facilitando la comprensión del flujo de datos.
2. Para aplicaciones web modernas, es aconsejable emplear un patrón de proxy (como se hizo con Node.js) para mediar entre el frontend y el servicio SOAP, evitando problemas de CORS y tipado complejo en el navegador.
3. Es fundamental definir correctamente los espacios de nombres (targetNamespace) en los contratos WSDL para evitar colisiones y asegurar que el servicio sea descubrible de forma estándar.

REFERENCIAS Y BIBLIOGRAFÍA

- [1] Tanenbaum, A.S. (2008). Sistemas distribuidos: principios y paradigmas. México. Pearson Educación.
- [2] Ceballos, F. J. (2006). Java 2, Curso de programación. México: Alfaomega, RaMa.
- [3] Deitel, H. M., & Deitel, P. J. (2004). Cómo programar en Java. México: Pearson Educación.
- [4] García Tomás, J., Ferrando, S., & Piattini, M. (2001). Redes para procesos distribuidos. México: Alfaomega Ra-Ma.
- [5] Fielding, R. (2000). Architectural Styles and the Design of Network-based Software Architectures. Dissertation. University of California, Irvine.

ANEXOS

Ejercicio 1: Conversor de Unidades (Java)

Interfaz del Conversor (ConversorAPI.java)

```
package e1;

import javax.ws.WebMethod;
import javax.ws.WebService;
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

```
@WebService(targetNamespace = "http://lab7.e1.conversor/")
public interface ConversorAPI {
    @WebMethod
    double cToF(double c);

    @WebMethod
    double fToC(double f);

    @WebMethod
    double mToFt(double m);

    @WebMethod
    double ftToM(double ft);

    @WebMethod
    double kgToLb(double kg);

    @WebMethod
    double lbToKg(double lb);
}
```

Implementación del Conversor (ConversorSOAP.java)

```
package e1;

import javax.ws.WebService;

@WebService(
    endpointInterface = "e1.ConversorAPI",
    serviceName = "ConversorSOAPService",
    portName = "ConversorSOAPPort",
    targetNamespace = "http://lab7.e1.conversor/"
)
public class ConversorSOAP implements ConversorAPI {

    @Override
    public double cToF(double c) {
        return ((c * 9.0) / 5.0) + 32.0;
    }

    @Override
    public double fToC(double f) {
        return ((f - 32.0) * 5.0) / 9.0;
    }

    @Override
    public double mToFt(double m) {
        return m * 3.28084;
    }

    @Override
    public double ftToM(double ft) {
        return ft / 3.28084;
    }

    @Override
    public double kgToLb(double kg) {
        return kg * 2.20462;
    }

    @Override
    public double lbToKg(double lb) {
        return lb / 2.20462;
    }
}
```


	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

Script Cliente Python (client.py)

```

from zeep import Client

WSDL_URL = "http://localhost:8080/conversor?wsdl"

def obtener_entrada(mensaje, tipo=float, obligatorio=True):
    while True:
        try:
            valor = input(mensaje).strip()
            if obligatorio and not valor:
                print("Error: Este campo es obligatorio.")
                continue
            return tipo(valor)
        except ValueError:
            etiqueta_tipo = {int: "entero", float: "decimal"}.get(tipo, "válido")
            print(f"Error: Por favor, ingrese un valor {etiqueta_tipo}.")

def main():
    print(f"\nConectando a: {WSDL_URL}")
    try:
        client = Client(WSDL_URL)
        print("Conexión establecida con éxito.\n")
    except Exception as e:
        print(f"Error al conectar: {e}")
        return

    while True:
        print("--- CONVERSION DE UNIDADES ---")
        print("1. Celsius a Fahrenheit")
        print("2. Fahrenheit a Celsius")
        print("3. Metros a Pies")
        print("4. Pies a Metros")
        print("5. Kilogramos a Libras")
        print("6. Libras a Kilogramos")
        print("7. Salir")

        opcion = input("\nSeleccione una opción (1-7): ").strip()

        if opcion == "1":
            valor = obtener_entrada("Grados Celsius: ")
            try:
                resultado = client.service.cToF(valor)
                print(f"Resultado: {valor}°C = {resultado:.2f}°F")
            except Exception as e:
                print(f"Error: {e}")
            print()

        elif opcion == "2":
            valor = obtener_entrada("Grados Fahrenheit: ")
            try:
                resultado = client.service.fToC(valor)
                print(f"Resultado: {valor}°F = {resultado:.2f}°C")
            except Exception as e:
                print(f"Error: {e}")
            print()

        elif opcion == "3":
            valor = obtener_entrada("Metros: ")
            try:
                resultado = client.service.mToFt(valor)
                print(f"Resultado: {valor} m = {resultado:.2f} ft")
            except Exception as e:
                print(f"Error: {e}")
            print()

        elif opcion == "4":
            valor = obtener_entrada("Pies: ")
            try:
                resultado = client.service.ftToM(valor)
                print(f"Resultado: {valor} ft = {resultado:.2f} m")
            except Exception as e:
                print(f"Error: {e}")
            print()

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

```

except Exception as e:
    print(f"Error: {e}")
    print()

elif opcion == "5":
    valor = obtener_entrada("Kilogramos: ")
    try:
        resultado = client.service.kgToLb(valor)
        print(f"Resultado: {valor} kg = {resultado:.2f} lb")
    except Exception as e:
        print(f"Error: {e}")
        print()

elif opcion == "6":
    valor = obtener_entrada("Libras: ")
    try:
        resultado = client.service.lbToKg(valor)
        print(f"Resultado: {valor} lb = {resultado:.2f} kg")
    except Exception as e:
        print(f"Error: {e}")
        print()

elif opcion == "7":
    print(";Hasta luego!")
    break
else:
    print("Opción no válida. Intente de nuevo.")

if __name__ == "__main__":
    main()

```

Ejercicio 2: Gestión de Tienda (Java + Python + Node.js)

Interfaz de la Tienda (SOAPImpl.java)

```

package e2.servicio.soap;

import e2.servicio.model.Item;
import java.util.List;
import javax.xml.ws.WebMethod;
import javax.xml.ws.WebService;

@WebService(targetNamespace = "http://lab7.e2.servicio.soap/")
public interface SOAPAPI {
    @WebMethod
    List<Item> getItems();

    @WebMethod
    String buyItem(String name, int cantidad);

    @WebMethod
    boolean addItem(Item item);

    @WebMethod
    boolean setItem(String name, int cantidad, double precio);

    @WebMethod
    boolean deleteItem(String name);
}

```

Implementación de la Tienda (SOAPImpl.java)

```

package e2.servicio.soap;

import e2.servicio.model.Item;
import java.util.List;
import javax.xml.ws.WebService;

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

```
@WebService(
    endpointInterface = "e2.servicio.soap.SOAPI",
    serviceName = "SOAPImplService",
    portName = "SOAPImplPort",
    targetNamespace = "http://lab7.e2.servicio.soap/"
)
public class SOAPImpl implements SOAPI {

    @Override
    public List<Item> getItems() {
        return Item.getItems();
    }

    @Override
    public String buyItem(String name, int cantidad) {
        return Item.buyItem(name, cantidad);
    }

    @Override
    public boolean addItem(Item item) {
        return Item.addItem(item);
    }

    @Override
    public boolean setItem(String name, int cantidad, double precio) {
        return Item.setItem(name, cantidad, precio);
    }

    @Override
    public boolean deleteItem(String name) {
        return Item.deleteItem(name);
    }
}
```

Modelo de Datos (Item.java)

```
package e2.servicio.model;

import java.io.Serializable;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Item implements Serializable {

    private static final long serialVersionUID = 1L;

    private String nombre;
    private int cantidad;
    private double costo;

    private static final List<Item> ITEMS = new ArrayList<>(
        Arrays.asList(
            new Item("Gaseosa", 15, 5.2),
            new Item("Galletas", 10, 1.6),
            new Item("Celular", 12, 900.0)
        )
    );

    public Item() {}

    public Item(String nombre, int cantidad, double costo) {
        this.nombre = nombre;
        this.cantidad = cantidad;
        this.costo = costo;
    }

    public static List<Item> getItems() {
        return ITEMS;
    }
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 12

```

public static boolean addItem(Item item) {
    if (
        item.nombre == null ||
        item.nombre.isEmpty() ||
        item.cantidad <= 0 ||
        item.costo <= 0
    ) {
        return false;
    }
    ITEMS.add(item);
    return true;
}

public static String buyItem(String name, int cantidad) {
    if (cantidad <= 0) {
        return "Invalid Quantity";
    }
    for (Item item : ITEMS) {
        if (item.nombre.equalsIgnoreCase(name)) {
            if (item.cantidad - cantidad < 0) {
                return "Not enough items";
            }
            item.cantidad -= cantidad;
            return (
                "Producto: " +
                item.nombre +
                " Cantidad: " +
                cantidad +
                " Total: " +
                String.format(java.util.Locale.US, "%.2f", (cantidad * item.costo))
            );
        }
    }
    return "Item no match";
}

public static boolean setItem(String name, int cantidad, double costo) {
    if (name == null || name.isEmpty() || cantidad <= 0 || costo <= 0) {
        return false;
    }
    for (Item item : ITEMS) {
        if (item.nombre.equalsIgnoreCase(name)) {
            item.cantidad = cantidad;
            item.costo = costo;
            return true;
        }
    }
    return false;
}

public static boolean deleteItem(String name) {
    if (name == null || name.isEmpty()) {
        return false;
    }
    return ITEMS.removeIf(item -> item.nombre.equalsIgnoreCase(name));
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public int getCantidad() {
    return cantidad;
}

public void setCantidad(int cantidad) {
    this.cantidad = cantidad;
}

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 13

```

public double getCosto() {
    return costo;
}

public void setCosto(double costo) {
    this.costo = costo;
}

@Override
public String toString() {
    return (
        "Item{nombre=" +
        nombre +
        ", cantidad=" +
        cantidad +
        ", costo=" +
        costo +
        "}"
    );
}
}

```

Proxy Backend (server.js)

```

const fastify = require("fastify")({ logger: true });

import { createClientAsync } from "soap";

fastify.register(require("@fastify/cors"), {
    origin: "*",
    methods: ["GET", "POST", "PUT", "DELETE"],
});

const SOAP_URL = "http://localhost:1516/WS/Store?wsdl";
let soapClientPromise;

async function getSoapClient() {
    if (!soapClientPromise) {
        soapClientPromise = createClientAsync(SOAP_URL);
    }
    return soapClientPromise;
}

function normalizeItems(result) {
    const raw = result?.return ?? [];
    const list = Array.isArray(raw) ? raw : [raw];
    return list.filter(Boolean).map((item) => ({
        nombre: String(item.nombre ?? ""),
        cantidad: Number.parseInt(item.cantidad ?? 0, 10),
        costo: Number.parseFloat(item.costo ?? 0),
    }));
}

fastify.get("/api/health", async () => ({ ok: true }));

fastify.get("/api/items", async () => {
    const client = await getSoapClient();
    const [result] = await client.getItemsAsync({});
    return normalizeItems(result);
});

fastify.post("/api/items", async (request) => {
    const { nombre, cantidad, costo } = request.body || {};
    const client = await getSoapClient();
    const [result] = await client.addItemAsync({
        arg0: { nombre, cantidad, costo },
    });
    return { ok: String(result?.return) === "true" };
});

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 14

```

fastify.put("/api/items/:nombre", async (request) => {
  const { nombre } = request.params;
  const { cantidad, costo } = request.body || {};
  const client = await getSoapClient();
  const [result] = await client.setItemAsync({
    arg0: nombre,
    arg1: cantidad,
    arg2: costo,
  });
  return { ok: String(result?.return) === "true" };
});

fastify.delete("/api/items/:nombre", async (request) => {
  const { nombre } = request.params;
  const client = await getSoapClient();
  const [result] = await client.deleteItemAsync({
    arg0: nombre,
  });
  return { ok: String(result?.return) === "true" };
});

fastify.post("/api/items/:nombre/buy", async (request) => {
  const { nombre } = request.params;
  const { cantidad } = request.body || {};
  const client = await getSoapClient();
  const [result] = await client.buyItemAsync({
    arg0: nombre,
    arg1: cantidad,
  });
  return { result: result?.return ?? "" };
});

const port = Number.parseInt(process.env.PORT || "3001", 10);
fastify.listen({ port, host: "0.0.0.0" }).catch((err) => {
  fastify.log.error(err);
  process.exit(1);
});

```

Script Cliente Python (client.py)

```

from typing import Any, Callable

from zeep import Client

WSDL_URL = "http://localhost:1516/WS/Store?wsdl"

def obtener_entrada(
    mensaje: str, tipo: Callable[[Any], Any] = str, obligatorio: bool = True
) -> Any:
    while True:
        try:
            valor = input(mensaje).strip()
            if obligatorio and not valor:
                print("Error: Este campo es obligatorio.")
                continue
            return tipo(valor)
        except ValueError:
            etiqueta_tipo = {int: "entero", float: "decimal"}.get(
                tipo,
                "válido",
            )
            print(f"Error: Por favor, ingrese un valor {etiqueta_tipo}.")

def main() -> None:
    print(f"\nConectando a: {WSDL_URL}")
    try:
        client = Client(WSDL_URL)
        print("Conexión establecida con éxito.\n")
    except Exception as e:

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 15

```

print(f"Error al conectar: {e}")
return

while True:
    print("--- GESTIÓN DE TIENDA ---")
    print("1. Ver inventario")
    print("2. Comprar producto")
    print("3. Agregar nuevo producto")
    print("4. Actualizar producto")
    print("5. Eliminar producto")
    print("6. Salir")

    opcion = input("\nSeleccione una opción (1-6): ").strip()

    if opcion == "1":
        try:
            items = client.service.getItems()
            if not items:
                print("\nEl inventario está vacío.")
            else:
                print(
                    "\n{:<20} | {:>8} | {:>12}".format(
                        "Producto", "Stock", "Precio ($/.)"
                    )
                )
                print("-" * 46)
                for item in items:
                    nombre = (
                        item["nombre"] if isinstance(item, dict) else item.nombre
                    )
                    cantidad = (
                        item["cantidad"]
                        if isinstance(item, dict)
                        else item.cantidad
                    )
                    costo = item["costo"] if isinstance(item, dict) else item.costo

                    print(
                        "{:<20} | {:>8} | {:>12.2f}".format(nombre, cantidad, costo)
                    )
                print()
            except Exception as e:
                print(f"\nError al obtener el inventario: {e}")
                print()

        elif opcion == "2":
            print("\n--- COMPRA ---")
            nombre = obtener_entrada("Nombre del producto: ")
            cantidad = obtener_entrada("Cantidad a comprar: ", int)
            try:
                resultado = client.service.buyItem(nombre, cantidad)
                print(f"Resultado: {resultado}")
            except Exception as e:
                print(f"Error: {e}")
                print()

        elif opcion == "3":
            print("\n--- NUEVO PRODUCTO ---")
            nombre = obtener_entrada("Nombre: ")
            cantidad = obtener_entrada("Stock inicial: ", int)
            costo = obtener_entrada("Precio: ", float)
            try:
                exito = client.service.addItem(
                    {"nombre": nombre, "cantidad": cantidad, "costo": costo}
                )
                if exito:
                    print(f"Producto '{nombre}' agregado correctamente.")
                else:
                    print(
                        f"No se pudo agregar '{nombre}'. Verifique si ya existe o si los datos son válidos."
                    )
            except Exception as e:
                print(f"Error: {e}")

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 16

```

print()

elif opcion == "4":
    print("\n--- ACTUALIZAR PRODUCTO ---")
    nombre = obtener_entrada("Nombre del producto a actualizar: ")
    cantidad = obtener_entrada("Nuevo stock: ", int)
    costo = obtener_entrada("Nuevo precio: ", float)
    try:
        exito = client.service.setItem(nombre, cantidad, costo)
        if exito:
            print(f"Producto '{nombre}' actualizado correctamente.")
        else:
            print(f"Producto '{nombre}' no encontrado.")
    except Exception as e:
        print(f"Error: {e}")
    print()

elif opcion == "5":
    print("\n--- ELIMINAR PRODUCTO ---")
    nombre = obtener_entrada("Nombre del producto a eliminar: ")
    try:
        exito = client.service.deleteItem(nombre)
        if exito:
            print(f"Producto '{nombre}' eliminado correctamente.")
        else:
            print(f"Producto '{nombre}' no encontrado.")
    except Exception as e:
        print(f"Error: {e}")
    print()

elif opcion == "6":
    print(";Hasta luego!")
    break
else:
    print("Opción no válida. Intente de nuevo.")

if __name__ == "__main__":
    main()

```